Lab Sheet — TP 3

Data Structures in Practice

Corresponds to: Lecture 4 (see TD 3)

Session Objectives

- Use heapq to solve a top-k problem efficiently.
- Build a dynamic array from scratch and observe amortised cost experimentally.
- Compare balanced vs. degenerate binary search tree performance.

Exercise 1 — Guided (done together in session)

Using a min-heap of bounded size k (via heapq), complete the function that returns the k largest elements seen in a stream, processing one element at a time (do not sort the whole stream).

```
import heapq

def top_k_largest(stream, k):
    heap = []
    for x in stream:
        if len(heap) < k:
            heapq.heappush(heap, x)
        else:
            # TODO: if x is larger than the current smallest kept element,
            # replace it (heapq.heappushpop is useful here)
            pass
    return sorted(heap, reverse=True)

print(top_k_largest([5, 1, 9, 3, 7, 2, 8], 3))   # expected [9, 8, 7]
```

Exercise 2 — Autonomous (in session, self-checked with the tests provided)

Implement a minimal dynamic array that doubles its internal capacity when full, then measure the time of each individual append over many operations to observe the amortised O(1) behaviour from TD3.

```
class DynamicArray:
    def __init__(self):
        self._capacity = 1
        self._size = 0
        self._data = [None] * self._capacity

    def append(self, x):
        # TODO: if self._size == self._capacity, double the capacity
        # by allocating a new list and copying existing elements
        # then store x at position self._size and increment self._size
        pass

arr = DynamicArray()
times = []
for i in range(20000):
    start = time.perf_counter()
    arr.append(i)
    times.append(time.perf_counter() - start)

plt.plot(times)
plt.xlabel('append number'); plt.ylabel('time (s)')
plt.title('Per-append cost: spikes at each resize')
plt.show()
```

Exercise 3 — Optional, to take further at home

Implement a simple binary search tree (insert and search only), then compare the average search time when (a) 5 000 random integers are inserted in random order, versus (b) the same 5 000 integers inserted already sorted (which produces a completely degenerate, list-like tree).